

NPTEL
NPTEL ONLINE COURSE

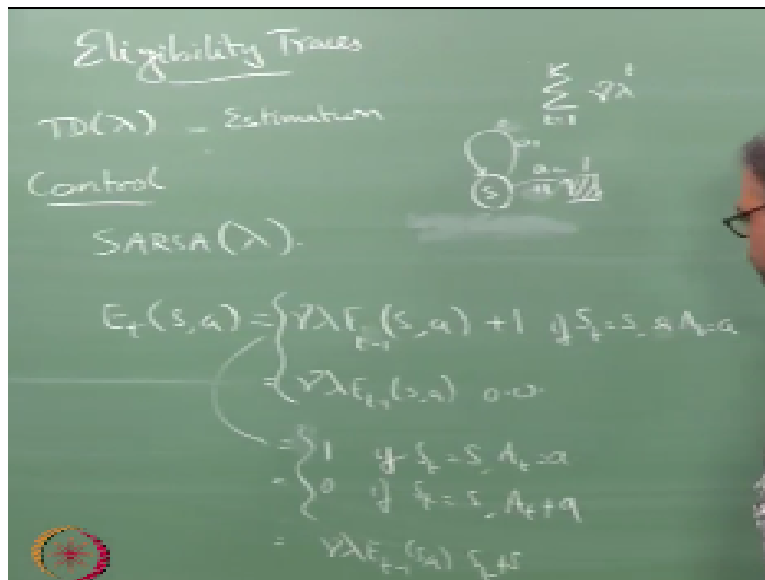
REINFORCEMENT LEARNING

Eligibility Trace Control

Prof. Balaraman Ravindran
Department of Computer Science and Engineering
Indian Institute of Technology Madras

So we were looking at eligibility traces.

(Refer Slide Time: 00:16)



Right we looked at $TD(\lambda)$, so what would be the next step to go, naturally would be to look at control right, after looking at $TD(\lambda)$ which is essentially a estimation method so we have to look at control so the easiest to think about in terms of control is version of SARSA that will use an

eligibility trace right so what does it mean to have eligibility trace in SARSA, yeah so the backward view is kind of easy to see where we are going with right so backward view in fact is almost like $TD(\lambda)$.

So essentially what I do now is I introduce a eligibility trace for, so I introduce an eligibility trace for state comma action pair just like you went from state values you go to action values for q learning and SARSA so you go to state comma action pairs you have an eligibility trace like this right and how do I update it, right this is exactly what we did there so this is kind of your accumulating trace.

So if you want replacing trace what you do okay, so there is one other thing you have to be careful about when you are talking about eligibility traces on state action pairs right suppose I am in some state s right this is action a_1 this is action a_2 I do not need even this, this is action a_2 that is action a_1 , this has a zero reward right, and then I have a policy right now that keeps taking action a_1 many many times and then finally takes action a_2 and gets a reward of $+1$ right if I am using accumulating traces so what will happen is every time I go around this loop I will keep adding one right.

The first time I go I will have one right the next time I go I will have $1 + \gamma \lambda$ the next time I go I will have $1 + \gamma \lambda$ this into $\gamma \lambda$ right so I will have $\gamma \lambda + \gamma \lambda$ the whole squared plus 1 okay I will keep going there at some and basically my trace for this will be some number of times $(\gamma \lambda)t$, right t running from 1 to sometime some k right depending on the exact values of $\gamma \lambda$ this number can be much larger than 1, right I have $\gamma \lambda$ plus $\gamma \lambda$ squared so for, yeah it is decaying right but you think about every time I am upping it by one rate so.

So it will be a number larger than one could be a number larger than one and what will be the trace of a_2 , when I finally take a_2 it will be one right and which action you think is going to get the higher fraction of this plus 1, a_1 will get updated more than a_2 right so what will happen next time I come to state s , again I will be more likely to take a_1 right again I will do this a few times and then I will take a_2 and again a_1 will get rewarded yes even if I use replacing trace what will happen that guy will get $\gamma \lambda$ okay it could still be a large number right.

So the rate at which my a_2 will actually come to beat my a_1 will be very very slow right but replacing trace will eventually fix your problem okay correct so accumulating traces are bad typically if you are doing control right it was not that much of an issue when we were talking about the estimation because I was not changing my actions based on this right but here I will be changing my actions based on the estimates I am making so the replacing, I mean accumulating traces are bad replacing traces are better right.

And you can do a slightly more aggressive version of replacing traces, what do I do in that case whenever I revisit a state and I take an action if it is for all the other actions that I could have taken from that state I make the trace 0, so I come to a state right I take an action from there so for all the other actions I could have taken from that state right I set the trace to zero that means if there is a specific reward I get by taking this action that will not go back to all the other actions I could have taken from that state right.

So to see what will happen suppose I come to s for the first time right and from here I take a_2 I have not taken a_1 so what will be the trace for a_1 , 0? I am taking only a_2 right so whatever reward I get here it will be, change the value of a_2 only it will not change the value of a_1 right, so I want that effect regardless of how many times I have visited state s before if I visit s right and then I take a_1 right then I come back to s after a while and then I take a_2 and I get some reward right so it is not likely that I would have gotten this reward by taking a_1 right.

Because it is a different completely different path I am choosing there so I do not want this reward to go update the value for a_1 right so what you can do is this is one way of doing this is accumulating trace this is replacing trace with three conditions, right this is a special case of replacing traces ok so you could do the regular replacing trace also wherein you would have decayed the value for all the other actions right this is one aggressive case of replacing traces so wherever you come back to a state for that action you set the trace to 1 for other actions you set the trace to 0.

So what would be the forward view for SARSA(λ), I mean ignoring this part, so what would be the equivalent we saw the forward view for TD (λ) what will be the equivalent forward view for

SARSA λ , it is the same thing you do not think too much about it right, so it is $1 - \lambda * \lambda$ weight for the first the first return one step return right and then λ squared weight for the two step return λ cube weight for the three step return and so on so forth ok do not worry do not do not do not think too much about it so that is basically it right.

(Refer Slide Time: 09:57)

$$\begin{aligned}
 E_t(s, a) &= \gamma \lambda E_{t-1}(s, a) + 1 \quad \text{if } s_t = s \text{ \& } A_t = a \\
 &= \gamma \lambda E_{t-1}(s, a) \quad \text{if } A_t \neq a \text{ \& } A_t = \arg\max_{a'} Q_t(s_t, a') \\
 &= 0 \quad \text{if } A_t \neq \arg\max_{a'} Q_t(s_t, a')
 \end{aligned}$$

$A_t = \arg\max_{a'} Q_t(s_t, a')$

And once you have this e_t , so what you do your $\Delta Q_t(s, a)$ will be some α times Δt into right where Δt is equal to, what, okay I do not even have to write these things down I mean you can generalize directly from TD(λ), almost everywhere I wrote v of s I have written $q(s, a)$, this I have just done the backward view the forward view we can likewise very easily write down the forward view for SARSA (λ).

Anyway good any questions on SARSA(λ) okay what is the next control algorithm we should look at, $Q \lambda$, yeah it turns out that there are many variants of $Q \lambda$ in the literature right so if you read the book you will get peng's $Q \lambda$ naive $Q \lambda$ and then you will also get something called Watkins' $Q \lambda$, Watkins' $Q \lambda$ that is the one we will look at very quickly because that is the one it is kind of you know justifiable even though in practice it does not work well right the other

things are just ad hoc stuff you know both peng's $Q \lambda$ and naive $Q \lambda$ which will be discussed in the first edition of the book.

They are pretty ad hoc right I mean it is not at all clear what they are converging to and there are equivalently many ways in which you can define a Q learning version for eligibility traces but what Watkins' $Q \lambda$ seems to be the most sensible way of doing things right even though it is, I mean empirically it can be very bad okay so if you think about what Watkins' $Q \lambda$, what would be the generalization or just if you think about Q learning itself what would be the generalization to using eligibility traces if you think of what we are doing in Q learning we are taking the immediate reward for taking an action.

And then assuming you are going to behave optimally thereafter right that is why I use the max and take the Q, so if I am going to use a two step return instead of a one step return what should be the two step return for Q learning, I should be behaving optimally subsequently right it cannot be a arbitrary action like in SARSA I never had that issue right so I could just take whatever action I did I could just use it right that is not an issue right but in Q learning it has to be the optimal next action right.

So we basically have this right so for this Δ is fine but when I am computing the return I have to make sure that I am actually taking the optimal action and sampling the rewards right how do I ensure that, I have to behave greedily see I cannot do ϵ greedy or anything like that if I do ϵ greedy then there is no guarantee that whatever sample I am drawing is going to be from the optimal policy when I mean there is no guarantee in the beginning when your value function is all messed up.

But even as learning progresses if I am exploring right the return I am generating will not be from the optimal policy right so as long as I am being greedy and generating my trajectories I can use the entire trajectory for Q learning right so if you think about it what I really need is okay I start from s I take some a that part is fine, right I go to some s' then what I really need to do is take $\max_a$, right I will go to some s'' and what I really need to do here is take $\max$ over a double prime and keep doing this right.

So if you think about it this is essentially the greedy policy right I am just executing the greedy policy if I execute the greedy policy then I can use how many ever steps I have gone I can use that entire return in my updation, no you would not follow the same trajectory always but we have other arguments about why we have to explore right we have looked at it all the while, I am just trying to build up why we need to be greedy first, but we all know we have to explore.

So we will have to figure out how to adjust that so we will come to the exploration part in a minute, right but people understand why I need the greedy trajectory because that is what the assumption of Q learning is right I am behaving greedily so like Subho pointed out and like we know from the Bandit times right we need to explore if you do not explore we will get stuck in something that is suboptimal right I cannot start regardless of where I can start with an arbitrary initialization of the Q values right.

But if I do not explore enough right I am going to get stuck in, get stuck in suboptimal policies right so I need to do exploration so how do I balance these two tensions, why yeah then your chain gets broken right because I want to find the return yeah that is because we are only doing one step right if I am going to take the return, the thing with λ is that I am taking some kind of a weighted average of one step return two step return three step return and four step return and so on so forth right.

So one way of thinking about doing this is to saying that I am going to keep track of the optimal action trajectory from every state I have visited imagine the amount of memory that you would need right I start from s , I take the optimal action go to s' , okay but then s' I explore right but then if I want to use the two step trajectory for updating s I will need to know what the optimal action from s' is, that I can do what about three step trajectory from s' , so what will I need I need to know the state I would have gone to from s' right and the greedy action I will have to take from that state.

Right I will have to remember all of that to get the three step trajectory for updating at s , that is too much work right, as soon as I explore at s' , all I can do there is use the greedy action from s' that is the longest trajectory that I can consider right update the value based on that

right and then zero out the eligibility trace I cannot do anything more with that, it does it does because as long as I do not explore if I do not explore for ten steps I can use from one step return to the ten step return, we are breaking the eligibility trace, that is the main problem with Watkins' $Q \lambda$.

So the eligibility trace will not run till the end of the trajectory so it will be only for short periods of time so that is a tradeoff that you are giving that is the tradeoff that you are taking, it is some form of λ return, if you try to write the forward view of Watkins $Q \lambda$ it turns out to be very messy expression because whenever you take an exploration only till that point you can do the averaging right so now what we say is that we do an averaging over all possible n step returns right and then all the weight is assigned to the last return that is what we are saying so now what happens is if I can do only up to some, say four step return after that I do exploration.

So for that particular state I will be doing an average over only the four returns right the state that came after that I will be doing the average over only three returns because any exploration, all the states that came before me I will make it 0 right and then the state just before that I will be doing the update for just one, right the state just before you do the exploration for example at s prime if you take an exploratory action then s prime will get the regular Q learning update, no λ business there right and I reset all the traces and start all over again.

So that is the tricky part with Watkins' $Q \lambda$ right and the forward view is incredibly hard to write so I am not even going to attempt to do that and do not worry I will not give it as homework as well maybe we will save it for an exam right so this is my same Δ , that we have in Q learning, the Δ does not change this update rule does not change right so this is exactly the same update rule that you have for SARSA what changes is this, if it is the greedy action right if the current action is the greedy action so not a , sorry if the current action is the greedy action right and s and a happens to be the current state in action I increment it by one right.

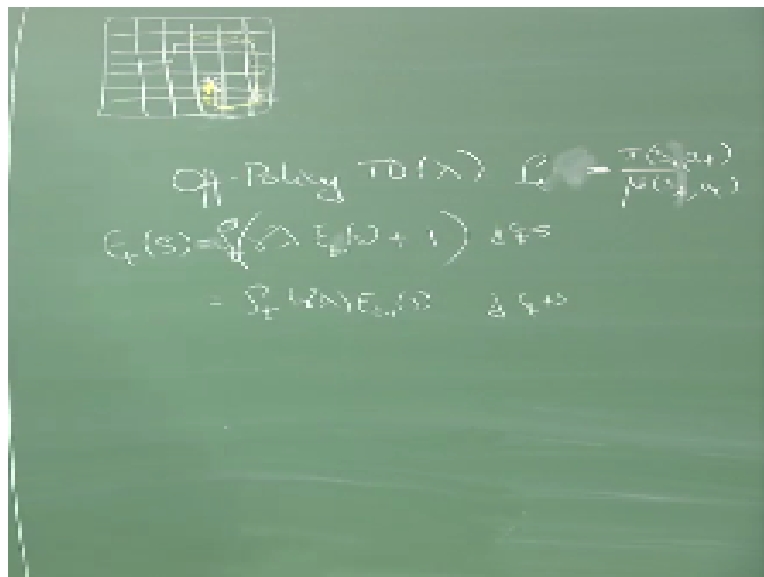
So this should be s t , sorry so if the current action is the greedy action but that is not the current action okay, a is not the current action then I decay the eligibility trace of a , I do not add a 1 right, if the current action is not the greedy action I zero out all the eligibility trace regardless of

what state what action everything, is it fine or should I do something else, what is exploring, this is the exploring part I have not written anywhere about action selection right so when you are doing the action selection you do it in the ϵ greedy fashion so whenever you actually pick an action that is not the greedy action that $1 - \epsilon$ times, you make this update to the eligibility trace.

If you have picked the greedy action then you do the regular accumulating trace update here so likewise you can write one version for replacing traces as well right or if you want to do Dutch traces you can write one version for dutch traces as well, dutch version here and a dutch version here so you could do all of that, ok, makes sense so the main problem with Watkins' $Q \lambda$ is all the traces will be very short unless you decrease your exploration by a significant amount right if you say I will explore with say ϵ of 0.1 or something then 10 percent of the times you are going to take exploratory actions right.

So things can get very, the traces will get chopped up a little and you lose the advantage of doing eligibility traces okay, so here is a here is a picture that they have maybe I should draw this on the board, let me try to draw this, it is easy enough okay.

(Refer Slide Time: 25:00)



Off-Policy TD(λ) $E_t = \frac{r(s_t, a_t) + \gamma V(s_{t+1}) - V(s_t)}{\lambda}$

$$E_t(s) = \sum_{i=0}^{\infty} \lambda^i E_t(s, i+1) \Delta F^s$$

$$= \sum_{i=0}^{\infty} \lambda^i \gamma^i E_t(s, i) \Delta F^s$$

So that is the final state I am going to get to okay and there is a reward let us say some reward say plus 10 everywhere else it is zero okay and I have say γ of 0.9 and λ of 0.9 okay so I am doing some random exploratory trajectory I go something like this I reached there and I reach the goal and I get the reward okay if I had been using regular Q learning or SARSA if I had been using regular Q learning or SARSA, which are the states that will get their values changed after one trajectory like this, online Q learning so this guy alone right this state alone the value of this action will go up right.

Value of that action will go up, so what happens in TD λ or SARSA λ in this case all the states but unfortunately no no SARSA λ probably it will stop somewhere there, this decays to such a small value after that I do not update it right so unfortunately you are getting wrong updates I mean the right action to do here is to probably go here but the point to notice with SARSA λ with one trajectory I update I know something or the other for so many states and actions.

While if I had done just plain SARSA or plain Q learning it would have been just for one step and that is one of the reasons we expect this to converge faster just to give you a very rough intuition as to why you expect this to converge faster this is this intuition comes from the backward view right because I am updating in so many states ok good so there are a couple of other things I am debating whether I should talk about or whether I should make you guys read about it, ok so what does Q learning do for us.

We keep calling Q learning as an off policy learning method right but it really does not fit into the structure of off policy algorithms that we saw in the Monte Carlo chapter right so we said we had one behavior policy and one estimation policy and we are trying to estimate the policy by behaving according to, estimate the value of that policy by behaving according to another policy and so on so forth, here we took a lot of shortcuts because the estimation policy was the optimal policy so we try to take all kinds of shortcuts right first of all we do not know what the optimal policy is right.

Right in the typical off policy learning, we talked about we have a π and a μ and then we did all of those things right now we do not know what the π is we know what the μ is because μ is what we

are behaving according to but π is the optimal policy I do not know what π is so really Q learning does not fall into the off policy mechanisms that we spoke about so you could think of having an off policy version of TD λ right.

So I could have a π which I am trying to evaluate but my trajectories could be coming to me from some μ right so how do I handle that, so showing the equivalence of SARSA and backward and forward view of SARSA λ you still have to assume that you are doing this offline not online so at the end of the trajectory only you are changing the values okay, that becomes very tricky you are never ever be running SARSA offline right because you want the values to be changing as you are executing so that you start learning right so you do not want to be executing random trajectories and then.

So it becomes a little tricky so there are more recent works where you can have an online version which allows you to which gives you equivalence between the online forward view and an online backward view anyway that is an aside so let us go back to off policy TD λ so what do you think we should do so we basically have this right, so that is the, alright so that is the update equation right the other one is $\gamma \lambda E_t - 1$, so I have to somehow figure out how will I, how am I going to insert my importance weight into this right I have to figure out how I am going to insert my importance weight into this how do you think I will insert my importance weight into this.

So let me define something that may can make my life little simpler I am going to define ρ_t of s as right whatever it is the π of whatever action I took at that time, yeah that is good point so I do not need to so it is a ρ_t is the importance weight at time t , it is for that particular choice right so at that time I made this choice right, I made a choice between some action at state s_t , I picked an action right what is the probability of me picking that if you remember our expression for the importance weight right.

We wrote it as a product of $\pi_{st} a_t$ summing over all t right so that is essentially that that item that is the entry I want so ρ_t will be $\pi_{st} a_t / \mu_{st} a_t$, that is one term from the importance weight if you if you actually have notes you can flip back and look at the important sampling thing that we

talked about right so I basically multiply this by ρt , but the question is what do I multiply by ρt , just the $\gamma \lambda E t - 1$, or should I multiply the $+1$ also right.

IIT Madras Production

Funded by

Department of Higher Education

Ministry of Human Resource Development

Government of India

www.nptel.ac.in

Copyrights reserved